

Search in Rotated Sorted Array

Company: Amazon.

Round: Online Assessment (OA)

Year: 2025

Difficulty: Medium

Topic: Arrays, Binary Search

Source: [LC33-Search in a rotated sorted array](#)

Problem Description

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index k ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Input Format

- An integer array `nums` containing distinct integers.
- An integer `target` representing the value to search for.

Output Format

- ☐ Return the index of target if it exists.
- ☐ Return -1 if target is not present.

Constraints

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- $-10^4 \leq \text{target} \leq 10^4$

Sample Input / Output

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3

Output: -1

Example 3:

Input: nums = [1], target = 0

Output: -1

Explanation

In **Example 1**, [1,2,3] is the smallest permutation, so the next permutation is obtained by making the smallest possible increase from the right side. Swapping 2 and 3 gives [1,3,2], which is the immediate next permutation in lexicographic order.

In **Example 2**, [3,2,1] is already the largest possible permutation because the elements are arranged in completely descending order. Therefore, there is no larger permutation, so the array is rearranged into the smallest possible order, [1,2,3].

In **Example 3**, [1,5,1] is obtained by finding that 1 at index 1 can be increased by swapping it with the greater element 5 on its right. This produces the next greater permutation [5,1,1].

Approach to Solve This Problem:

The key idea is to use **binary search** while identifying the sorted half of the rotated array. Since the array is rotated, the entire array is not sorted, but at every step, at least one half of the current search space is guaranteed to be sorted. We use this sorted half to determine whether the target can exist there and eliminate the other half.

1. **Find the middle element:**
Calculate the middle index using $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$. If $\text{nums}[\text{mid}]$ is equal to the target, return mid because the target has been found.
2. **Identify the sorted half:**
Check whether $\text{nums}[\text{left}] \leq \text{nums}[\text{mid}]$. If this condition is true, the left half is sorted. Otherwise, the right half is sorted.
3. **Check the left sorted half:**
If the left half is sorted, check whether the target lies between $\text{nums}[\text{left}]$ and $\text{nums}[\text{mid}]$. If it does, search in the left half by moving right to $\text{mid} - 1$. Otherwise, search in the right half by moving left to $\text{mid} + 1$.
4. **Check the right sorted half:**
If the right half is sorted, check whether the target lies between $\text{nums}[\text{mid}]$ and $\text{nums}[\text{right}]$. If it does, search in the right half by moving left to $\text{mid} + 1$. Otherwise, search in the left half by moving right to $\text{mid} - 1$.

5. **Continue until the target is found:**

Repeat the above process while left <= right. If the search space becomes empty without finding the target, return -1.

Java Solution:

```
import java.util.*;

public class Main {

    public static void nextPermutation(int[] nums) {
        int n = nums.length;

        // 1. Find the pivot
        int i = n - 2;

        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }

        // 2. Find the successor and swap
        if (i >= 0) {
            int j = n - 1;

            while (nums[j] <= nums[i]) {
                j--;
            }

            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }

        // 3. Reverse the suffix
        int start = i + 1;
        int end = n - 1;

        while (start < end) {
```

```
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] nums = new int[n];

        for (int i = 0; i < n; i++) {
            nums[i] = sc.nextInt();
        }

        nextPermutation(nums);

        for (int i = 0; i < n; i++) {
            System.out.print(nums[i] + " ");
        }

        sc.close();
    }
}
```

Solution:

- **Initialize the search range:** Set `left = 0` and `right = n - 1` to represent the current search space.
- **Find the middle element:** Calculate `mid = left + (right - left) / 2`. If `nums[mid] == target`, return `mid` because the target has been found.
- **Identify the sorted half:** Check whether `nums[left] <= nums[mid]`. If true, the left half is sorted; otherwise, the right half is sorted.
- **Search in the left sorted half:** If the left half is sorted, check whether `target` lies between `nums[left]` and `nums[mid]`. If `nums[left] <= target < nums[mid]`, move `right` to `mid - 1` to continue searching in the left half. Otherwise, move `left` to `mid + 1` to search in the right half.
- **Search in the right sorted half:** If the right half is sorted, check whether `target` lies between `nums[mid]` and `nums[right]`. If `nums[mid] < target <= nums[right]`, move

`left` to `mid + 1` to search in the right half. Otherwise, move `right` to `mid - 1` to search in the left half.

- **Return the result:** Continue the process while `left <= right`. If the search range becomes empty without finding the target, return `-1`.

Complexity Analysis:

Time Complexity: $O(\log n)$

In every iteration, we identify the sorted half and eliminate approximately half of the remaining search space. Therefore, the total number of iterations is logarithmic with respect to n .

Space Complexity: $O(1)$

The algorithm uses only a constant number of variables such as `left`, `right`, and `mid`. No additional array or data structure is required, so the solution uses constant extra space.
